

PRACTICAL WEEK 9

1. Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical9.1.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

Write the code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <time.h>

#define BUFFER_SIZE 4096

int main(int argc, char *argv[])
{
    if (argc != 3)
    {
        printf("Usage: %s <source> <destination>\n", argv[0]);
        return 1;
    }

    int source = open(argv[1], O_RDONLY);

    if (source < 0)
    {
        perror("open source");
        return 1;
    }

    int destination = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination < 0)
    {
        perror("open destination");
        close(source);
        return 1;
    }

    char buffer[BUFFER_SIZE];
    ssize_t bytes_read;
    struct timespec start, end;

    clock_gettime(CLOCK_MONOTONIC, &start);

    while ((bytes_read = read(source, buffer, BUFFER_SIZE)) > 0)
    {
        ssize_t total_written = 0;

        while (total_written < bytes_read)
        {
            ssize_t bytes_written = write(
                destination,
                buffer + total_written,
                bytes_read - total_written
```

```

    );

    if (bytes_written < 0)
    {
        perror("write");
        close(source);
        close(destination);
        return 1;
    }

    total_written += bytes_written;
}

if (bytes_read < 0)
{
    perror("read");
}

clock_gettime(CLOCK_MONOTONIC, &end);

double elapsed = (end.tv_sec - start.tv_sec) +
                 (end.tv_nsec - start.tv_nsec) / 1000000000.0;

off_t position = lseek(source, 0, SEEK_CUR);

printf("Low-level I/O copy completed\n");
printf("Bytes copied: %ld\n", (long)position);
printf("Execution time: %.6f seconds\n", elapsed);

close(source);
close(destination);

return 0;
}

```

Standard Library File Copy:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical9.1.standard.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |

```

Write the code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define BUFFER_SIZE 4096

int main(int argc, char *argv[])
{
    if (argc != 3)
    {
        printf("Usage: %s <source> <destination>\n", argv[0]);
        return 1;
    }

    FILE *source = fopen(argv[1], "rb");

    if (source == NULL)
    {
        perror("fopen source");
        return 1;
    }

    FILE *destination = fopen(argv[2], "wb");

    if (destination == NULL)
    {
        perror("fopen destination");
        fclose(source);
        return 1;
    }

    char buffer[BUFFER_SIZE];
    size_t bytes_read;
    size_t total_bytes = 0;

    struct timespec start, end;

    clock_gettime(CLOCK_MONOTONIC, &start);

    while ((bytes_read = fread(buffer, 1, BUFFER_SIZE, source)) > 0)
    {
        size_t bytes_written = fwrite(buffer, 1, bytes_read, destination);

        if (bytes_written != bytes_read)
        {
            perror("fwrite");
            fclose(source);
            fclose(destination);
            return 1;
        }
    }
}
```

```

        return 1;
    }

    total_bytes += bytes_written;
}

clock_gettime(CLOCK_MONOTONIC, &end);

double elapsed = (end.tv_sec - start.tv_sec) +
    (end.tv_nsec - start.tv_nsec) / 1000000000.0;

printf("Standard library copy completed\n");
printf("Bytes copied: %zu\n", total_bytes);
printf("Execution time: %.6f seconds\n", elapsed);

fclose(source);
fclose(destination);

return 0;
}

```

Create a test file: run this

```
echo "This is a test file for Practical 9.1." > source.txt
```

check:

```
cat source.txt
```

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ echo "This is a test file for Practical 9.1." > source.txt
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ cat source.txt
This is a test file for Practical 9.1.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |

```

Run the program:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical9.1 source.txt copy.txt
Low-level I/O copy completed
Bytes copied: 39
Execution time: 0.000037 seconds

```

Verify the both copies:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ cat copy.txt
This is a test file for Practical 9.1.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ cmp source.txt copy.txt
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ls -l source.txt copy.txt
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 39 Oct  6 14:02 copy.txt
-rw-r--r-- 1 vishwa_teja_veerandi vishwa_teja_veerandi 39 Oct  6 13:58 source.txt
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |

```

Testing:

```

vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ dd if=/dev/zero of=largefile.bin bs=1M count=100
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0.0590026 s, 1.8 GB/s
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical9.1 largefile.bin
Usage: ./practical9.1 <source> <destination>
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |

```

2. Develop a program that redirects standard input and output using file descriptors and dup2(). Demonstrate how shells implement I/O redirection.

Create a file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical9.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

Write the code:

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int input_fd;
    int output_fd;

    input_fd = open("input.txt", O_RDONLY);

    if (input_fd < 0)
    {
        perror("open input.txt");
        return 1;
    }

    output_fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (output_fd < 0)
    {
        perror("open output.txt");
        close(input_fd);
        return 1;
    }

    if (dup2(input_fd, STDIN_FILENO) < 0)
    {
        perror("dup2 stdin");
        return 1;
    }

    if (dup2(output_fd, STDOUT_FILENO) < 0)
    {
        perror("dup2 stdout");
        return 1;
    }

    close(input_fd);
    close(output_fd);

    char buffer[1024];
    ssize_t bytes_read;

    while ((bytes_read = read(STDIN_FILENO, buffer, sizeof(buffer))) > 0)
    {
        write(STDOUT_FILENO, buffer, bytes_read);
    }

    return 0;
}
```

Create a input file:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ echo "Hello from Practical 9.2" > input.txt
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ cat input.txt
Hello from Practical 9.2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano practical9.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ echo "Hello from Practical 9.2" > input.txt
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ cat input.txt
Hello from Practical 9.2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc practical9.2.c -o practical9.2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./practical9.2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ cat output.txt
Hello from Practical 9.2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ |
```